

Efficient Vision Architectures

Samsung Research Institute, Bangalore

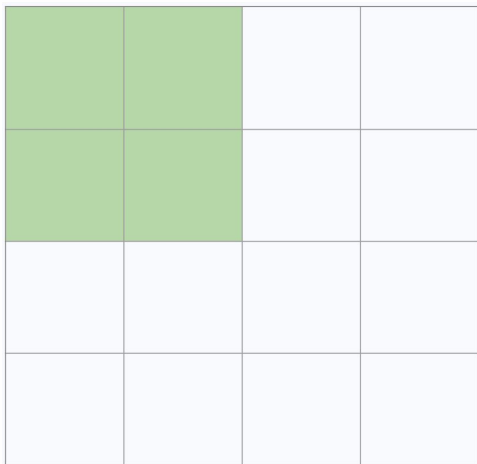
15th June, 2026



The “Problem with ViT”

Attention module time complexity scales quadratically with number of patches (directly dependent on image resolution).

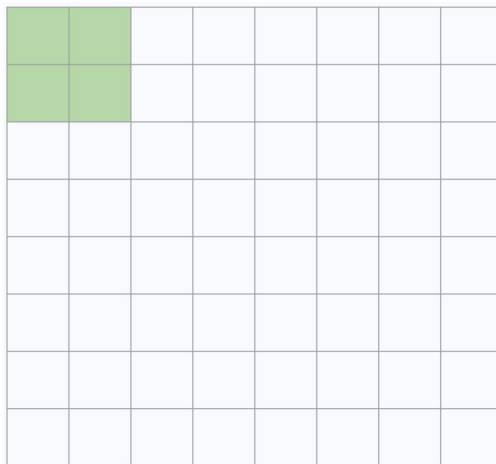
Image size: 4 x 4



Patches = $2 \times 2 = 4$

Time Complexity = $4 \times 4 = 16$

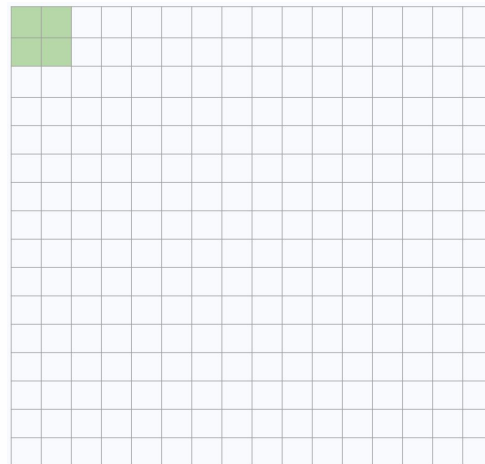
Image size: 8 x 8



Patches = $4 \times 4 = 16$

Time Complexity = $16 \times 16 = 256$

Image size: 16 x 16



Patches = $8 \times 8 = 64$

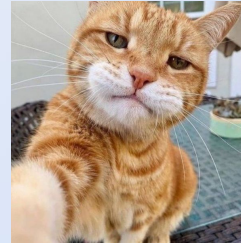
Time Complexity = $64 \times 64 = 4096$

Question: Why not just resize on move on??

512 x 512



128 x 128



Answer

Tasks like **Semantic Segmentation** and **Medical Anomaly Detection** where fine pixel level details are important will suffer with performance degradation.

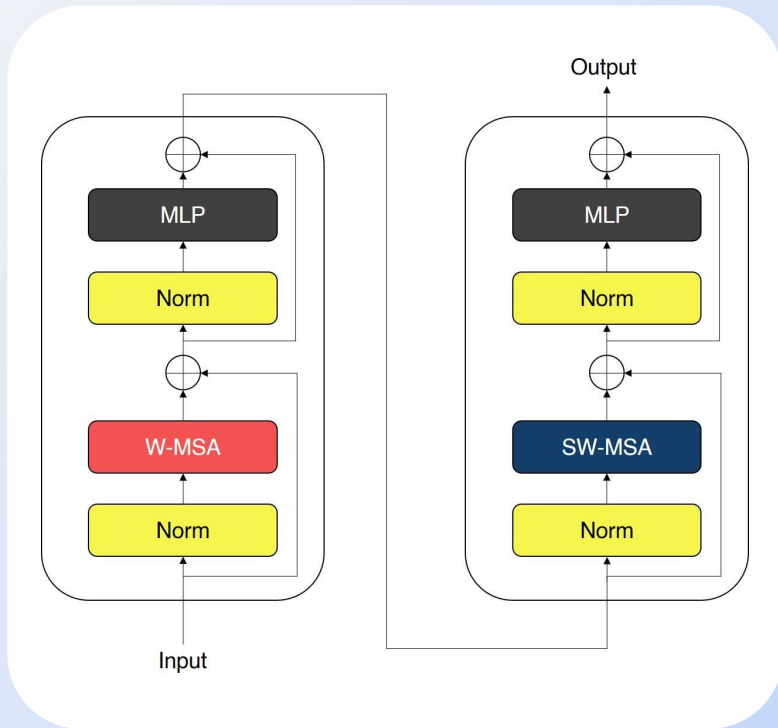


How to reduce time complexity while still being able to process high resolution images ?

Swin-Transformer

- Swin → Shifted Window.
- Reduces the attention complexity from $O(N^2)$ to Linear.
- Main components:
 1. **Shifted Window Attention**
 2. **Hierarchical Feature Maps.**
- Can serve as a general purpose vision backbone.

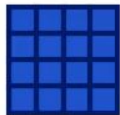
(Introduced by Microsoft Research)



Shifted Window Attention

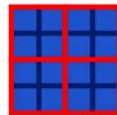
Standard MSA

Attention for each patch is computed against all patches, resulting in quadratic complexity

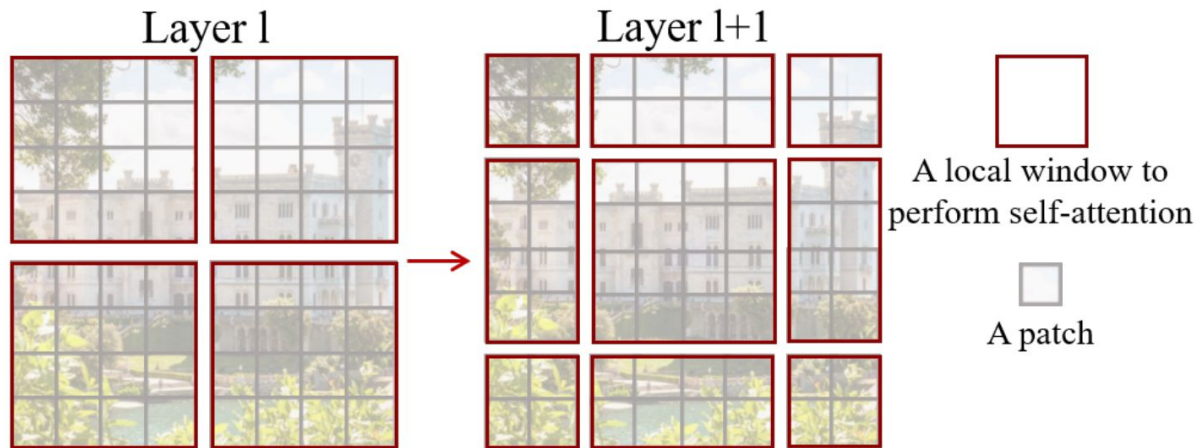


Window-based MSA

Attention for each patch is only computed within its own window (drawn in red). Window size is 2x2 in this example.



Shifted Window Attention



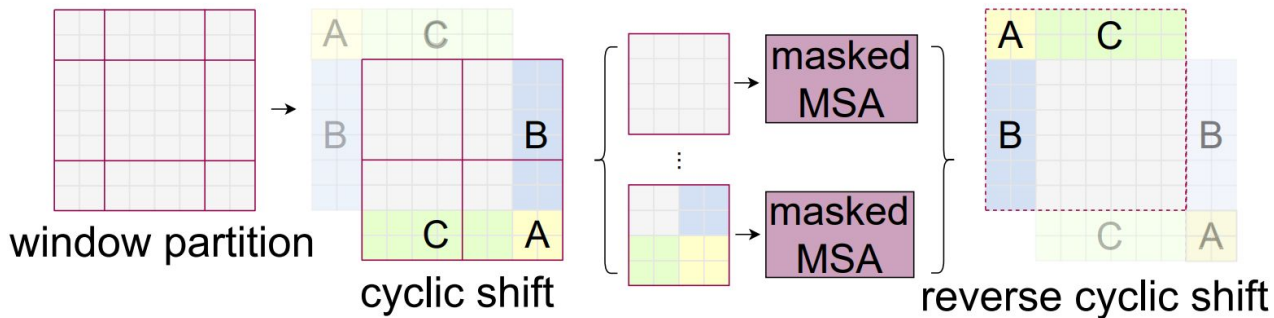
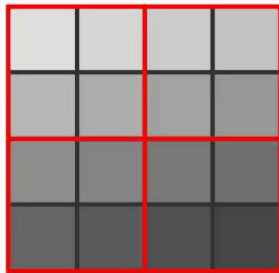
Cyclic Shift and Attention Masking in MSA

Shifted Window MSA

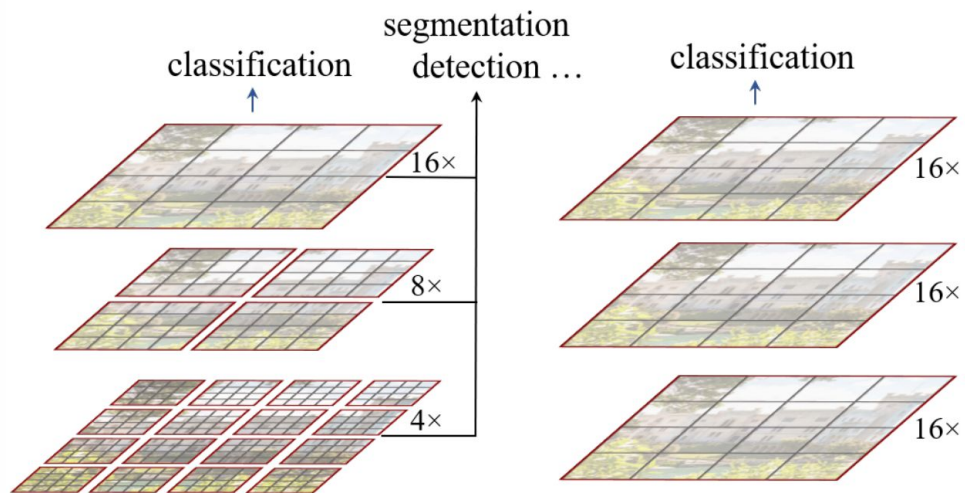
Step 1: Shift window by a factor of $M/2$, where M = window size

Step 2: For efficient batch computation, move patches into empty slots to create a complete window.

This is known as 'cyclic shift' in the paper.



Hierarchical Feature Maps



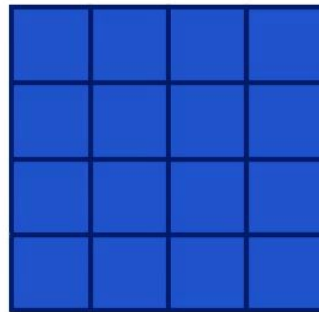
Patch Merging

Assuming that $n=2$, and each group consists of 2×2 neighboring patches

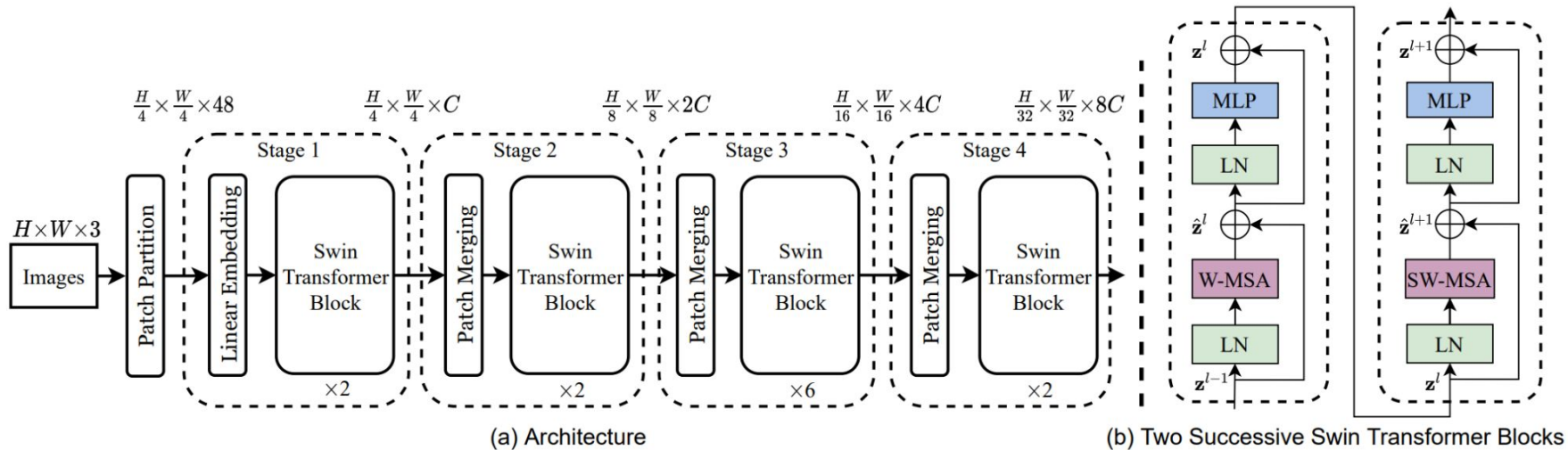
Step 1: Split input image into groups of 2×2

Step 2: In each group, stack the patches depth-wise

Step 3: Combine the stacked groups



Overall Architecture



$$\begin{aligned}\hat{z}^l &= \text{W-MSA}(\text{LN}(z^{l-1})) + z^{l-1}, \\ z^l &= \text{MLP}(\text{LN}(\hat{z}^l)) + \hat{z}^l, \\ \hat{z}^{l+1} &= \text{SW-MSA}(\text{LN}(z^l)) + z^l, \\ z^{l+1} &= \text{MLP}(\text{LN}(\hat{z}^{l+1})) + \hat{z}^{l+1},\end{aligned}$$

$$\begin{aligned}\Omega(\text{MSA}) &= 4hwC^2 + 2(hw)^2C, \\ \Omega(\text{W-MSA}) &= 4hwC^2 + 2M^2hwC,\end{aligned}$$

*Let's explore some **Elastic** architectures..*

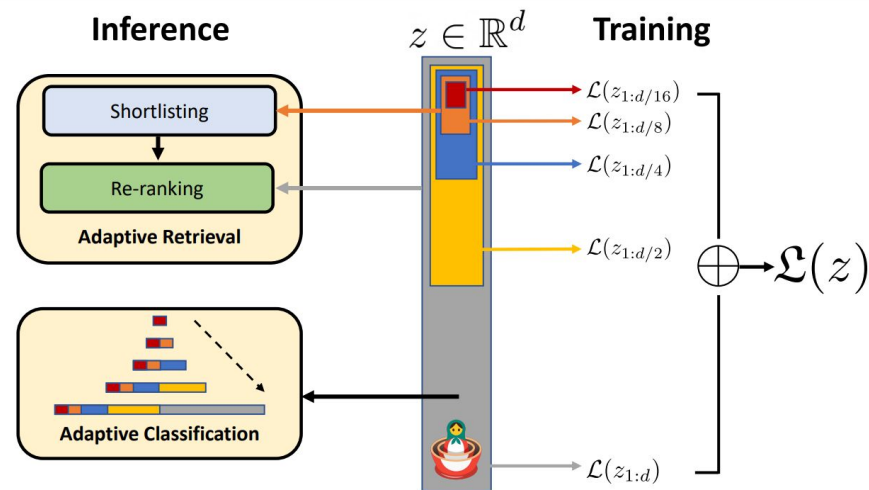
MRL (Matryoshka Representation Learning):

Comes from the concept of Matryoshka Dolls.



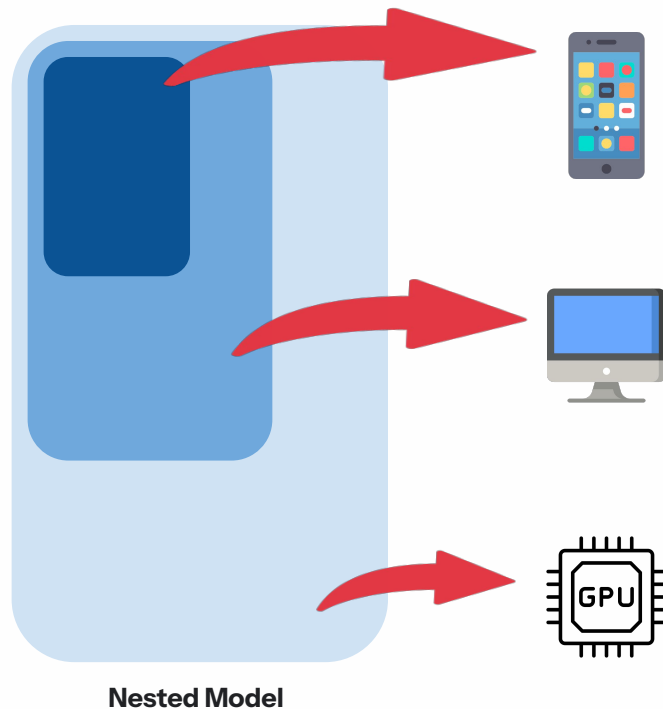
MRL (Matryoshka Representation Learning):

- Feature vectors or embedding (learned representations) are used for many downstream tasks.
- The deployment of representations has two steps:
(1) an expensive yet constant-cost forward pass to compute the representation and,
(2) utilization of the representation for downstream applications.
- The size of the embedding directly affects the cost of the downstream task.
- In other words, the permissible size of the embedding is determined by the available computational resources.
- MRL learns representations of varying capacities within the same high-dimensional vector and allows a single embedding to adapt to the computational constraints of downstream tasks.
- The smaller nested embeddings are as good as independently trained embeddings.
- Only a single embedding is stored.



Matformer / MatViT:

- Models are needed to be deployed in various setting with varying inference constraints.
- Hardware capacity ranges from low capacity mobile devices to massive GPU clusters.
- We can not train a separate model for every setting. Fewer models are provided my model families.
- Matformer solves this issue by incorporating a nested FFN structure within a standard Transformer model.
- This gives us:
 - 1. Lesser Memory Footprint** (No need to store multiple models of different sizes).
 - 2. Huge number of models at inference time.**
 - 3. Single training** (No separate trainings for different models).
- AI



Matformer / MatViT:

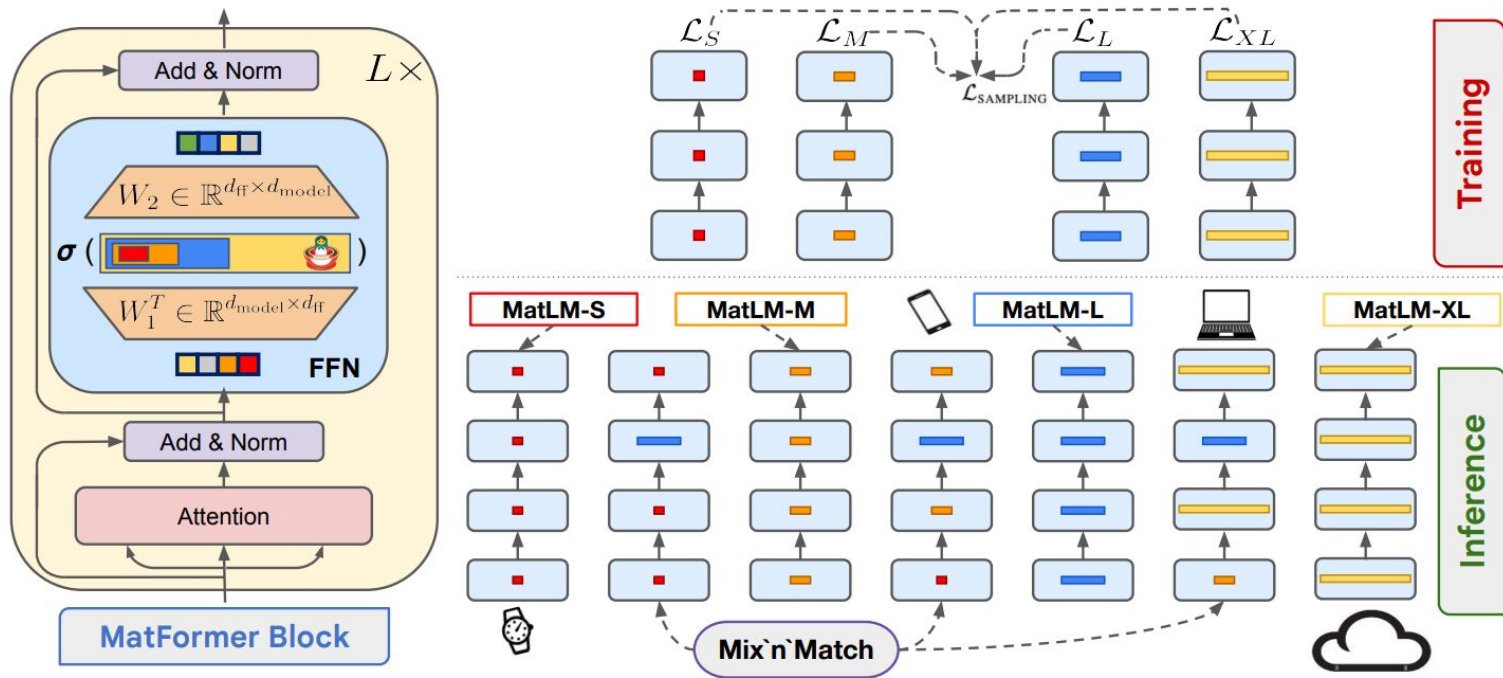
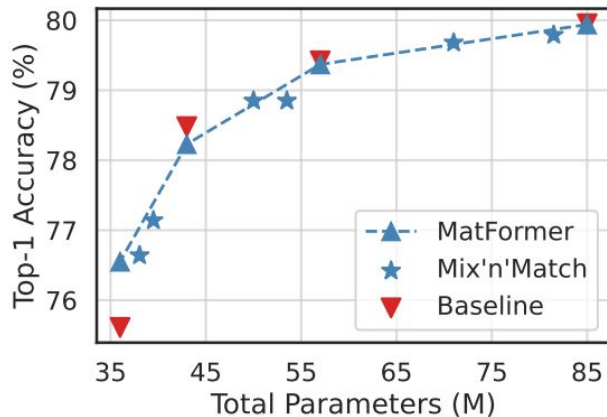


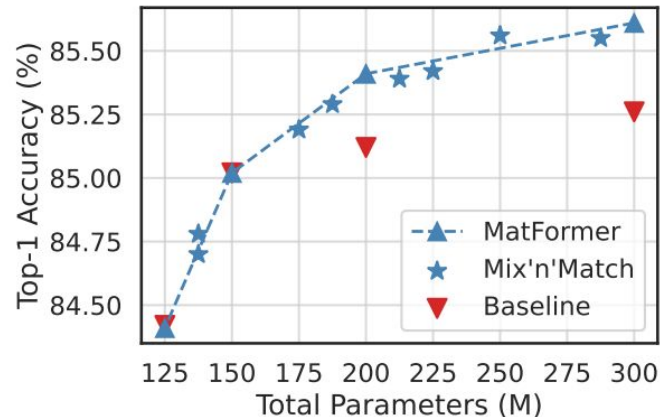
Figure 1: MatFormer introduces nested structure into the Transformer's FFN block & trains all the submodels, enabling free extraction of hundreds of accurate submodels for elastic inference.

Compute - Performance Tradeoff

Metric	Full MatViT-B	Smallest Sub-model	Separately Trained Small ViT
Parameters	86M	22M	22M
Top-1 Acc (INet)	84.5%	81.2%	81.4%
Retrieval mAP	0.76	0.72	0.04



(a) B/16 trained on ImageNet-1K with AugReg



(b) L/16 pretrained on IN-21K → ImageNet-1K.

Similar works to check:

- FlexTron
- Dynamic-ViT
- EAViT
- SparseViT
- FlexiViT

Thank you



